sensors

MDPI

Article

Trajectory Tracking Controller for Quadrotor by Continual Reinforcement Learning in Wind-Disturbed Environment

Yanhui Liu, Lina Hao *, Shuopeng Wang and Xu Wang

School of Mechanical Engineering and Automation, Northeastern University, Shenyang 110819, China; 2370189@stu.neu.edu.cn (Y.L.); 2210118@stu.neu.edu.cn (S.W.); 2300456@stu.neu.edu.cn (X.W.)
* Correspondence: haolina@me.neu.edu.cn

Abstract

The extensive deployment of quadrotors in complex environmental missions has revealed a critical challenge: degradation of trajectory tracking accuracy due to time-varying wind disturbances. Conventional model-based controllers struggle to adapt to nonlinear wind field dynamics, while data-driven approaches often suffer from catastrophic forgetting that compromises environmental adaptability. This paper proposes a reinforcement learning framework with continual adaptation capabilities to enhance robust tracking performance for quadrotors operating in dynamic wind fields. We develop a continual reinforcement learning framework integrating continual backpropagation algorithms with reinforcement learning. Initially, a foundation model is trained in wind-free conditions. When wind disturbance intensity undergoes gradual variations, a neuron utility assessment mechanism dynamically resets inefficient neurons to maintain network plasticity. Concurrently, a multi-objective reward function is designed to improve both training precision and efficiency. The Gazebo/PX4 simulation platform was utilized to validate the wind disturbance stepwise growth and stochastic variations. This approach demonstrated a reduction in the root mean square error of trajectory tracking when compared to the standard PPO algorithm. The proposed framework resolves the plasticity loss problem in deep reinforcement learning through structured neuron resetting, significantly enhancing the continual adaptation capabilities of quadrotors in dynamic wind fields.

Keywords: quadrotor; deep reinforcement learning; continual learning; trajectory tracking control

check for updates

Academic Editor: Xiyuan Chen

Received: 3 July 2025
Revised: 27 July 2025
Accepted: 6 August 2025
Published: 8 August 2025

Citation: Liu, Y.; Hao, L.; Wang, S.; Wang, X. Trajectory Tracking Controller for Quadrotor by Continual Reinforcement Learning in Wind-Disturbed Environment. *Sensors* **2025**, *25*, 4895. https://doi.org/10.3390/s25164895

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the rapid advancement of the unmanned aerial vehicle (UAV) domain, the quadrotor—owing to its distinctive characteristics—has been extensively employed in various UAV control systems [1–5]. The simple structure, high stability, strong maneuverability, rapid response, and exceptional payload adaptability of the system allow for high-precision execution of tasks, including aerial hovering and trajectory tracking [6–9]. However, in practical applications, quadrotors are often confronted with challenges, including the difficulty of accurately modeling wind dynamics, uncertainty in environmental parameters, and inherently nonlinear dynamical equations [10,11]. Thus, it is both critical and urgent to deepen our understanding of UAV adaptability, robustness, and intelligence under environmental disturbances.

Outdoor wind disturbances represent a significant factor affecting the stability and performance of quadrotor control systems. To achieve precise trajectory tracking of a quadrotor in the presence of such disturbances, existing control approaches under complex

environmental conditions can be broadly categorized into model-based and data-driven strategies. Model-based methods explicitly incorporate aerodynamic disturbance terms into the quadrotor's dynamical equations and design disturbance-rejection controllers to enhance tracking accuracy [12,13]. In [14], a robust H_∞ guaranteed cost controller is developed for quadrotors, concurrently addressing model uncertainties and external disturbances to effectively attenuate perturbation effects. In [15], a sliding-mode controller is combined with a fixed-time disturbance observer to design a controller for the position–attitude system, thereby handling time-varying wind disturbances in trajectory tracking. In [16], a model predictive control-based position controller is integrated with an SO(3)-based nonlinear robust attitude control law to counteract external disturbances. Model-based controllers largely depend on accurate dynamical models of wind disturbances and can deliver stable and reliable performance under low uncertainties. Nonetheless, these controllers often exhibit limited adaptability when handling complex nonlinear and multivariable coupling effects, and their parameter tuning presents significant difficulties.

Compared to model-based control strategies, data-driven approaches demonstrate enhanced adaptive capabilities when addressing complex nonlinear, time-varying, or unknown disturbances [17–19]. Regarding data-driven state observers for quadrotors, ref. [20] employed Koopman operator theory to construct dynamics in two distinct environments and integrated them with an MPC controller for trajectory tracking. Ref. [21] proposed a reinforcement learning (RL) framework incorporating external force compensation and a disturbance observer to attenuate wind gust effects during trajectory tracking. In [22], a novel wind perturbation estimator utilizing neural networks and cascaded Lyapunov functions was used to derive a full backstepping controller for quadrotor trajectory tracking. Beyond observer design, deep reinforcement learning (DRL) methods enable direct policy learning through reward function design, allowing quadrotors to interact with wind-disturbed environments. In [23], an RL-based controller was trained to directly generate desired three-axis Euler angles and throttle commands for disturbance rejection. Ref. [24] proposes a distributed architecture utilizing multi-agent reinforcement learning to reduce user perception delay and minimize UAV energy consumption. Ref. [25] integrated policy relief and significance weighting into incremental DRL to enhance control accuracy in dynamic environments. In [26], dual-critic neural networks supplant the conventional actor–critic framework to approximate solutions to Hamilton–Jacobi–Bellman equations for disturbance-robust quadrotor tracking. However, such data-driven methods typically employ static neural networks with frozen weights post-training. This approach relies on wind-condition-specific datasets during offline training, limiting generalization capability beyond the training distribution. Moreover, when environmental dynamics drift, conventional online fine-tuning strategies are highly susceptible to catastrophic forgetting.

To address trajectory tracking control for quadrotors under significantly varying wind disturbances and mitigate catastrophic forgetting during continual learning, this paper proposes a continual reinforcement learning framework. This framework resolves the inability of RL policies to adapt to substantial environmental changes during quadrotor trajectory tracking in wind-disturbed environments. Most existing continual learning methods operate at either the parameter level [27,28] or the functional output level [29], focusing on global connection strength or output similarity. Continual backpropagation (CBP) [30] employs selective resetting of inefficient neurons at the structural level, continuously activating dormant units while maintaining an effective representation rank without additional loss terms, old model storage, or parameter importance evaluation, thereby enabling lightweight continual learning. As illustrated in Figure 1, our framework first trains a foundation trajectory tracking policy using proximal policy optimization (PPO) in wind-free conditions. Subsequently, it integrates CBP-based continual reinforcement

learning to adapt this foundation policy to progressively evolving wind fields. By enhancing neural network plasticity through CBP, the framework improves quadrotor adaptation in dynamic environments. Additionally, a novel reward function is designed to enhance policy accuracy and effectiveness. In summary, this work investigates quadrotor trajectory tracking in disturbed environments via continual deep reinforcement learning. The primary contributions are summarized as follows:

1. We propose a novel continual reinforcement learning framework for quadrotor trajectory tracking, which significantly enhances adaptability in continuously varying wind fields.
2. We analyze the characteristics of tracking control and design an innovative reward function to accelerate network training convergence and improve control precision.
3. We validate the proposed framework on a quadrotor simulation platform. Experimental results demonstrate that our method achieves high trajectory tracking accuracy both in wind-free conditions and under gradually increasing wind disturbances, evidencing its strong learning capability.

The remainder of this paper is organized as follows. Section 2 formulates the trajectory tracking problem and elaborates on the reward function design. Section 3 introduces the proposed continual reinforcement learning framework and its core mechanisms. Section 4 presents comprehensive experimental results with comparative analysis under wind-disturbed conditions. Finally, Section 5 concludes this paper and discusses future research directions.

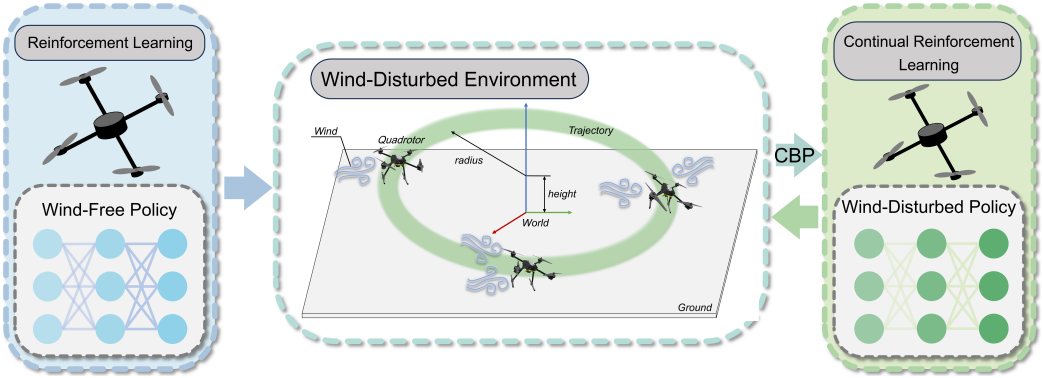

Figure 1. Overall architecture of the proposed continual reinforcement learning framework for quadrotor trajectory tracking under wind disturbance environment.

2. Preparation

2.1. Representation of System and Action

The quadrotor is a canonical underactuated system capable of generating six-degree-of-freedom (6-DOF) motion through only four independent control inputs. The relationship between the quadrotor's body frame and the inertial frame is illustrated in Figure 2. The time evolution of the quadrotor's flight state can be described by differential equations. The position of the quadrotor in the inertial frame is denoted as $\boldsymbol{p}^e = [p_x, p_y, p_z]^{\mathrm{T}}$, and $\dot{\boldsymbol{p}}^e = [v_x, v_y, v_z]^{\mathrm{T}} \in \mathbb{R}^3$ denotes the velocity vector expressed in the inertial frame. The quadrotor's attitude is represented by Euler angles, whose time derivatives are governed by the kinematic relationship $\dot{\boldsymbol{\Theta}} = \boldsymbol{W} \cdot \boldsymbol{\omega}^b \in \mathbb{R}^3$ with

$$\dot{\boldsymbol{\Theta}} = [\dot{\alpha} \quad \dot{\beta} \quad \dot{\gamma}]^{\mathrm{T}}, \quad \boldsymbol{\omega}^b = [\omega_{xb} \quad \omega_{yb} \quad \omega_{zb}]^{\mathrm{T}} \tag{1}$$

In Equation (1), $\boldsymbol{\Theta}$ denotes the Euler angles, $[\alpha \quad \beta \quad \gamma]$ denotes roll, pitch, and yaw, respectively, and $\boldsymbol{\omega}^b \in \mathbb{R}^3$ represents the angular velocity in the body frame. The coordinate transformation matrix $\boldsymbol{W} \in \mathbb{R}^{3\times3}$ is defined as follows:

$$\boldsymbol{W} = \begin{bmatrix} 1 & \tan\beta \sin\alpha & \tan\beta \cos\alpha \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha/\cos\beta & \cos\alpha/\cos\beta \end{bmatrix} \tag{2}$$

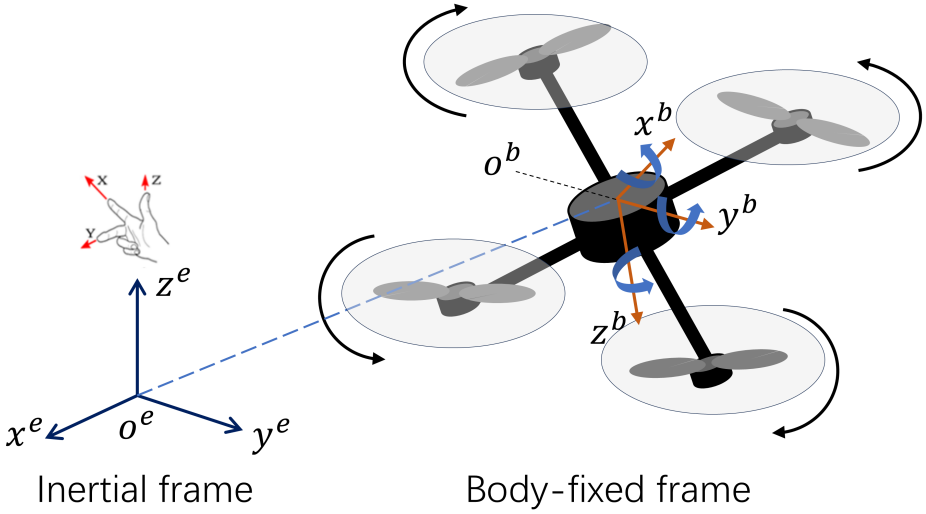

Figure 2. Structure illustration of a quadrotor.

This study designs a deep reinforcement learning-based control policy that enables the quadrotor to perform precise trajectory tracking tasks. In the tracking scenario, the motion relationship between the quadrotor and the reference trajectory is critical for controller design. We define a 9-dimensional state vector at time t as

$$S_t^* = \{\Delta p_x, \Delta p_y, \Delta p_z, \Delta\alpha, \Delta\beta, \Delta\gamma, \Delta v_x, \Delta v_y, \Delta v_z\}^T \in \mathbb{R}^9 \tag{3}$$

The state vector S_t^* can be decomposed into three subset vectors. $\Delta \boldsymbol{p}_t^e = [\Delta p_x, \Delta p_y, \Delta p_z]^T$ represents the relative distance between the quadrotor's current position and the target position in three coordinates. $\Delta\boldsymbol{\Theta}_t = [\Delta\alpha, \Delta\beta, \Delta\gamma]^T$ denotes the relative difference between the quadrotor's Euler angles and the target attitude. The relative velocity between the quadrotor and the target is represented as $\Delta \boldsymbol{v}_t^e = [\Delta v_x, \Delta v_y, \Delta v_z]^T$. The dynamic behavior of the quadrotor over time, its motion trends, and the environmental context enable the neural network to better comprehend the system and generate effective control strategies for the quadrotor. Considering the kinematic relationship between the quadrotor and target states, this study defines the state observation for DRL as the current state concatenated with the states from the previous n timesteps, formulated as follows:

$$S_t = \{S_t^*, S_{t-1}^*, S_{t-2}^*, \ldots, S_{t-n}^*\} \in \mathbf{R}^{9(n+1)} \tag{4}$$

The position of the quadrotor can be controlled using its inertial-frame velocity components v^e and yaw angle γ. Consequently, the action space for the reinforcement learning agent is defined as $u_t = [v_x, v_y, v_z, \gamma] \in \mathbb{R}^4$, where $\{v_x, v_y, v_z\}$ represent the inertial-frame velocities and γ denotes the yaw angle.

2.2. Design of Reward Function

A well-designed reward function facilitates faster convergence and superior performance during the reinforcement learning process. By analyzing the characteristics of trajectory tracking control tasks across different stages and integrating state observations, this study designs the reward function as follows:

$$r_t = -(\omega_1 \|\Delta \boldsymbol{p}_t^e\| + \omega_2 \|\Delta \boldsymbol{\Theta}_t\| + \omega_3 \|\Delta \boldsymbol{v}_t^e\| + \omega_4 \|u_t - u_{t-1}\|) + r_p^+ + r_{crash}^- \tag{5}$$

Error terms related to the quadrotor's state constitute penalty components. In Equation (5), $\{\omega_1, \omega_2, \omega_3, \omega_4\}$ denote the weighting coefficients for the position penalty, attitude penalty, velocity penalty, and control smoothness penalty, respectively. Reward sparsity is a common challenge in reinforcement learning environments for quadrotor trajectory tracking tasks [31]. To address this, sparse reward terms $\{r_p^+, r_{crash}^-\}$ are incorporated into the reward function. r_p^+ represents a positive sparse reward for positional accuracy, granted when $\|\Delta \boldsymbol{p}_t^e\| \leq d$. r_{crash}^- denotes a significant negative sparse penalty for critical failures (e.g., crashes or exceeding operational boundaries), serving to constrain abnormal quadrotor behavior. The inclusion of sparse rewards mitigates the agent's over-reliance on local reward signals, thereby reducing the risk of overfitting and accelerating the learning process.

3. Continual Reinforcement Learning Method

3.1. Problem Description

In this study, environmental variations are formulated as a sequence of temporally ordered subtasks. The state transition function drifts during environmental changes while the state and action spaces remain invariant. Let $\mathcal{D} = [E_1, \ldots, E_{h-1}, E_h, \ldots]$ denote the dynamic environment set, where E_h represents specific characteristics of the dynamic environment at the h-th stage. The primary objective of our reinforcement learning model is to derive an optimal control policy that maximizes the expected reward $J(\theta)$ under network parameters θ. The expected long-term return is expressed as follows:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)}[r(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta(\tau)}\left[\sum_{i=0}^{\infty} \zeta^i r_i\right] \tag{6}$$

where $\tau = \{s_0, a_0, s_1, a_1, \ldots\}$ denotes a complete episode trajectory, r_i represents the instantaneous reward after agent–environment interaction, and ζ is the discount factor. According to the policy gradient theorem, the expectation of $r(\tau)$ under the distribution $\pi_\theta(\tau)$ is given by

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)}[r(\tau) \nabla \log \pi_\theta(\tau)] \tag{7}$$

During the learning process, model parameters are optimized through policy approximation, expressed as follows:

$$\theta_{t-1}^* = \arg\max_{\theta \in \mathbb{R}^d} J_{E_{h-1}}(\pi_\theta) \tag{8}$$

The external disturbance problem refers to situations involving environmental transitions. The goal of continual learning is to enable autonomous updating from θ_{t-1}^* to θ_t^* when environmental characteristics transition to E_h:

$$\theta_t^* = \arg\max_{\theta \in \mathbb{R}^d} J_{E_h}(\pi_\theta) \tag{9}$$

3.2. The PPO Algorithm

Compared to value-based methods, policy-based reinforcement learning offers distinct advantages by directly learning the policy itself, thereby accelerating the learning process and enhancing decision-making performance in continuous action spaces. PPO integrates policy gradients with importance sampling to address the inherent sample inefficiency of basic policy gradient methods. Utilizing an actor–critic architecture, the PPO-based quadrotor controller design is illustrated in Figure 3.

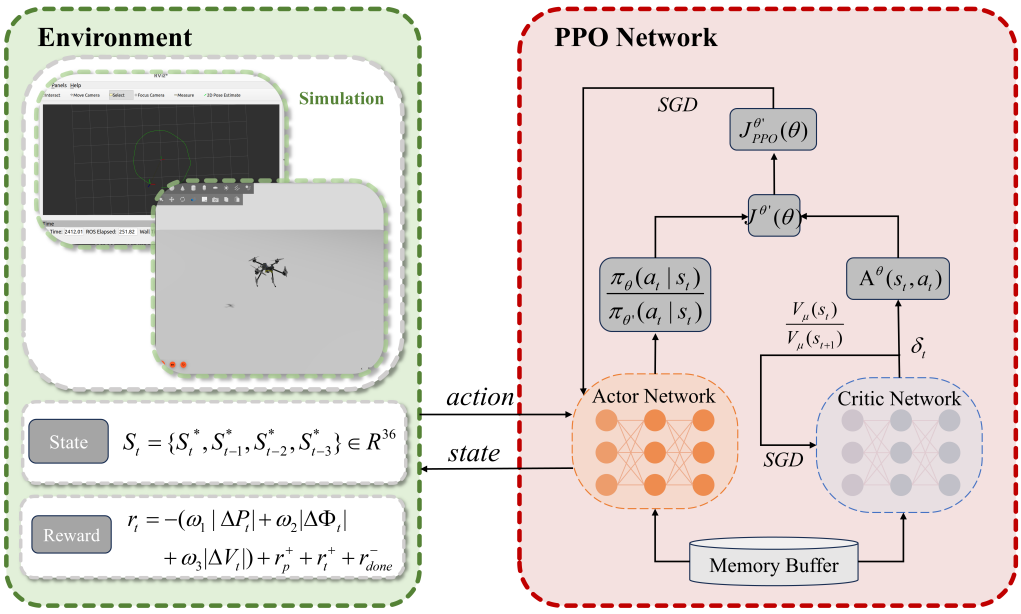

Figure 3. Quadrotor trajectory tracking control based on PPO.

PPO employs generalized advantage estimation (GAE), which balances the trade-offs between Monte Carlo methods and single-step temporal difference (TD) approaches through weighted fusion of multi-step TD errors. The GAE formulation is defined as follows:

$$A^\theta(s_t|a_t) = A_{GAE}^\theta = \sum_{l=0}^{T-t} (\zeta\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \zeta V(s_{t+1}) - V(s_t) \tag{10}$$

In Equation (10), λ serves as a trade-off parameter regulating the weighting distribution of multi-step returns, δ_t denotes the single-step TD error, and $V(s_t)$ represents the state-value function. This design enables the advantage function to capture global information from multi-step interactions while mitigating the high variance inherent in Monte Carlo methods.

To enhance policy update stability and prevent excessive fluctuations that could destabilize optimization, the algorithm incorporates the advantage function $A^\theta(s_t, a_t)$ and importance sampling into the objective function (6), yielding the surrogate objective function:

$$J^{\theta'}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)} A^{\theta'}(s_t|a_t)\right] \tag{11}$$

In Equation (11), θ' denotes the behavioral policy parameters interacting with the environment, while θ represents the optimization parameters. The objective $J^{\theta'}(\theta)$ is computed via the product of the advantage $A^\theta(s_t, a_t)$ (evaluated from samples s_t, a_t) and the tractable importance ratio $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)}$. To maintain proximity between the behavioral

policy $\pi_{\theta'}(a_t|s_t)$ and optimized policy $\pi_\theta(a_t|s_t)$ during importance sampling, we enforce policy constraints through a clipping function, resulting in the final objective:

$$J^{\theta'}_{PPO}(\theta) = \sum_{(s_t,a_t)} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)} A^{\theta'}(s_t,a_t), \text{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\right) A^{\theta'}(s_t,a_t)\right) \quad (12)$$

By integrating generalized advantage estimation and a clipping mechanism, PPO effectively balances bias–variance trade-offs during policy optimization while preventing abrupt policy updates. These properties establish PPO as an ideal solution for trajectory tracking in quadrotors, particularly when addressing continuous action spaces and multiple disturbance scenarios.

3.3. CBP Method

In dynamic environments, quadrotors must continuously adapt to fluctuating wind conditions. While traditional deep learning approaches perform well in static settings, they suffer from loss of plasticity in continual learning scenarios [30]. To enhance the agent's adaptability to dynamic environments and facilitate efficient exploration, this study integrates continual backpropagation (CBP) into the PPO framework. Unlike conventional approaches that freeze network weights after training, CBP reinitializes neurons at the structural level to preserve architectural diversity. Figure 4 illustrates the operational principle of CBP within the neural network architecture.

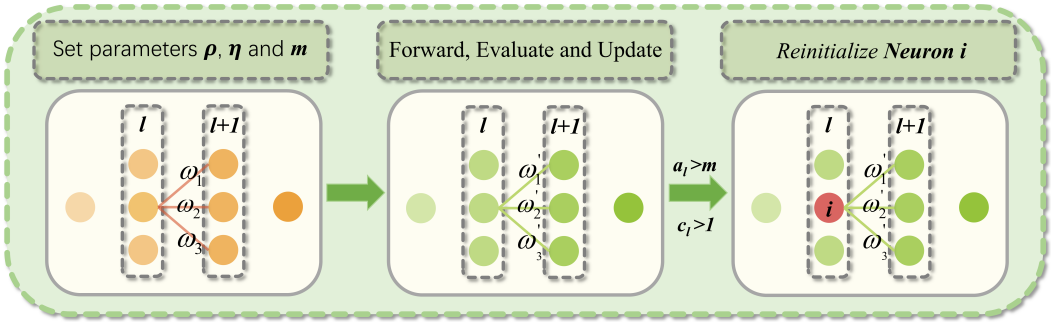

Figure 4. Continual backpropagation mechanism.

The principal advantage of CBP lies in its unit utility assessment and selective reinitialization mechanism. The utility metric for hidden unit i in layer l quantifies its contribution to downstream layers via an exponentially weighted moving average:

$$u^l_i = \eta \cdot u^l_i + (1-\eta) \cdot |h^l_{i,t}| \cdot \sum_{k=1}^{n_{l+1}} |w^l_{i,k}| \quad (13)$$

where η denotes the decay rate balancing historical and current contributions, $h^l_{i,t}$ represents the activation of unit i in layer l at time t, and $w^l_{i,k}$ denotes the weight connecting unit i in layer l to unit k in layer $l+1$.

When resetting a hidden unit, CBP initializes all outgoing weights to zero to prevent perturbation of learned network functions. However, this zero-initialization renders new units immediately nonfunctional and vulnerable to premature resetting. To mitigate this, newly initialized units receive a grace period of m updates during which they are reset-exempt. Only after exceeding this maturity threshold m are units considered mature. Subsequently, at each update step, a fraction ρ of mature units per layer undergo reinitialization. In practice, ρ is set to an extremely small value, ensuring approximately one unit replacement per several hundred updates.

During each network parameter update, CBP introduces only two additional operations compared to standard forward/backward propagation: unit utility updates and neuron resets. The unit utility update applies an exponential moving average to every neuron in each hidden layer, involving one absolute-value summation and one multiply–accumulate per weight. Its time complexity is $O\left(\sum_l n_l \cdot n_{l+1}\right)$, which is on the same order as the total number of network parameters. Neuron resets select and reinitialize a constant number of mature units at a very low rate ρ; because this occurs infrequently and affects only a fixed number of units per event, its overall overhead is negligible. In terms of memory overhead, CBP allocates for each hidden unit an additional utility value u_i^l and age a_i^l, totaling $O\left(\sum_l n_l\right)$. Thus, CBP maintains continuous structural variation and network plasticity with minimal extra time and space costs, fully compatible with standard neural network training pipelines.

Through CBP integration, the quadrotor UAV iteratively updates network parameters to maintain adaptability in complex, dynamically evolving environments.

3.4. Integrated Method

Reinforcement learning experiments exhibit greater stochasticity and variance compared to supervised learning due to inherent algorithmic randomness and sequential data dependencies influenced by the learning process itself [32,33]. Figure 5 illustrates quadrotor flight dynamics under varying wind conditions: stable flight occurs under low winds; moderate winds induce attitude deviations and instability; and high winds cause significant flight disruption or failure.

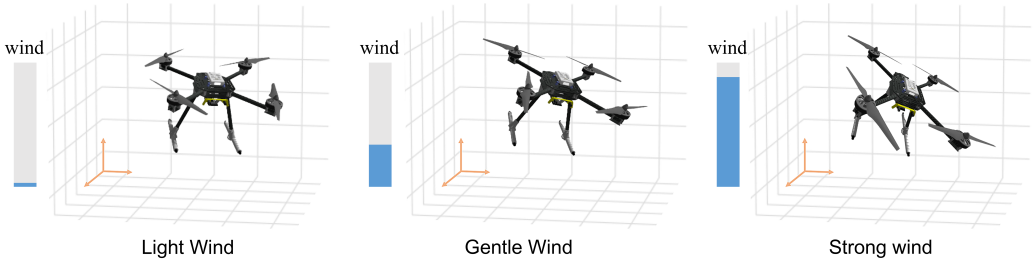

Figure 5. Quadrotor flight behavior under different wind conditions.

A foundation policy π_θ trained in environment E_{h-1} exhibits degraded performance when deployed in altered environment E_h, necessitating adaptive capability. This study proposes a continual reinforcement learning framework where wind-free control strategies serve as foundational models, enabling policy adaptation to environmental changes. By integrating CBP's continual learning mechanism into PPO's policy gradient updates, our approach maintains learned policy retention while flexibly generating new action distributions, achieving rapid adaptation to dynamic wind fields.

Algorithm 1 details the continual reinforcement learning process for updating quadrotor control policy π_θ in dynamic environments. Wind disturbance signals serve as environmental variables during trajectory tracking. The policy π_θ pretrained in E_{h-1} serves as the foundation model, where the agent learns through standard policy gradients in E_h while CBP concurrently performs structural neuron updates. After each full network update, neurons meeting reinitialization criteria are reset. Since the agent has acquired fundamental representations of the E_{h-1} state–action space, the CBP algorithm enhances network plasticity to facilitate adaptation to E_h variations.

Algorithm 1: Continual reinforcement learning for UAV control.

Input: Environment set $\mathcal{D}$, PPO hyperparameters, CBP parameters (ρ, η, m)
Output: Optimized policy π_θ for wind-disturbed environments
Data: Actor network π_θ, Critic network V_ϕ
for *episode* $i = 1$ **to** MaxEpisode **do**
  Collect trajectory $\tau = \{s_t, a_t\}$ via π_θ
  Calculate PPO objective $J^{\text{CLIP}}(\theta)$ (Equation (12))
  Actor Update:
    $\theta \leftarrow \theta + \alpha_{\text{actor}} \nabla_\theta J^{\text{CLIP}}(\theta)$
  Critic Update:
    $\phi \leftarrow \phi - \alpha_{\text{critic}} \nabla_\phi \left(V_\phi(s_t) - V^{\text{target}}\right)^2$
  Apply CBP Mechanism:
  for *layer* $l = 1$ **to** L **do**
    Compute layer activations h_l via forward pass
    Update utility metrics:
      $u_i^l \leftarrow \eta \cdot u_i^l + (1 - \eta) \cdot \left|h_{l,i}\right| \cdot \sum_k \left|w_{l,i,k}\right|$ (Equation (13))
    Increment age: $a_i^l \leftarrow a_i^l + 1$
    if $a_i^l > m$ **then**
      Determine replacement count $n_{\text{replace}} = \lfloor \rho \cdot N_{\text{mature}} \rfloor$
      Identify n_{replace} units with minimal u_i^l
      Reinitialize weights: $w_{l,i,\cdot} \sim \mathcal{N}(0, \sigma)$
      Reset $a_i^l \leftarrow 0$, $u_i^l \leftarrow 0$
    end
  end
end

4. Experiment and Discussion

4.1. Experimental Setup and Parameter Configuration

The training process was executed on an Ubuntu 22.04 system equipped with an AMD Ryzen 7 6800H CPU and NVIDIA RTX 3050 GPU. Our continual reinforcement learning control algorithm was implemented in Python 3.10 using PyTorch 2.4, with the Adam optimizer employed for network training [34]. We established a simulation environment integrating Gazebo 2 with the PX4 flight controller, leveraging Gazebo's high-fidelity physics engine to simulate quadrotor flight under wind disturbances [35]. The parameters of the quadrotor are shown in Table 1. The learning framework follows the architecture depicted in Figure 1.

Table 1. Parameters of the quadrotor.

Parameter	Value
Mass of the quadrotor	2 kg
Principal moment of inertia	$(0.0217,\ 0.0217,\ 0.04)\ \text{kg} \cdot \text{m}^2$
Motor constant	$8.549 \times 10^{-6}\ \text{kg} \cdot \text{m}$
Moment constant	$0.016\ \text{kg} \cdot \text{m}^2$
Wheelbase	0.348 m

The PPO implementation utilizes separate actor and critic networks, each containing four hidden layers. The actor network processes state observations $S_t \in \mathbb{R}^{36}$ as input and outputs system actions $u_t \in \mathbb{R}^4$, with hidden layer dimensions of 64, 128, 256, and 64 neurons, respectively. The critic network accepts system states and actions as input and estimates state value $V(S_t)$, featuring hidden layers of 128, 256, 256, and 128 neurons. All hidden layers employ ReLU activation functions.

Reward function weights were empirically determined, with state error components defined as penalty terms to promote stable flight control. Given the critical importance of position and attitude control in trajectory tracking, larger weights were assigned to position ($\omega_1 = 0.12$) and attitude ($\omega_2 = 0.12$) penalties, while velocity and control smoothness penalties received lower weights ($\omega_3 = 0.05, \omega_4 = 0.1$). To enhance exploration and simplify training, two sparse reward components were implemented: a positional accuracy reward $r_p^+ = 9$ triggered when $\|\Delta \boldsymbol{p}_t^e\| < d = 0.05$ m and a critical failure penalty $r_{crash}^- = -100$ imposed for crashes or boundary violations.

The operational boundary for trajectory tracking was defined as a 5 m × 5 m × 5 m cubic volume. Additional training parameters, referenced from prior works [25,36], are detailed in Table 2.

Table 2. Training parameters.

Parameter	Value
GAE bias–variance (λ)	0.95
Actor learning rate	2×10^{-4}
Critic learning rate	3×10^{-4}
Policy update epochs	9
Initial action standard deviation	0.4
Action std decay rate	2×10^{-4}
Learning rate decay coefficient	10^{-4}
Discount factor (ζ)	0.98
CBP replacement rate (ρ)	10^{-5}
CBP utility decay rate (η)	0.99
CBP maturity threshold (m)	50

To evaluate the performance of the continual reinforcement learning-based quadrotor control strategy in windy environments, this study conducts trajectory tracking experiments under both wind-free and wind-disturbed conditions. The root mean square error (RMSE) of tracking deviation serves as the quantitative evaluation metric for assessing the tracking control performance of the proposed method. Let M denote the total number of samples and e_i represent the positional tracking error of the i-th sample. The RMSE is calculated as follows:

$$\text{RMSE} = \sqrt{\frac{1}{M}\sum_{i=1}^{M} e_i^2} \quad \forall i \in A \tag{14}$$

4.2. Performance Analysis in the Wind-Free Environment

The reward function design incorporates not only the current flight state but also historical states from the preceding n timesteps to capture motion trends and environmental context for policy learning. Figure 6 illustrates the variation in control error RMSE and

maximum error with different history lengths n. The curves demonstrate significant error reduction as n increases from 0 to 3. For $n > 3$, errors exhibit a slight increase, indicating that excessive historical information introduces redundant noise. Consequently, this study selects $n = 3$ as an optimal compromise between information sufficiency and network complexity.

Under wind-free conditions, we trained the quadrotor's trajectory tracking policy using the PPO algorithm over 2000 training episodes. Figure 7 shows the corresponding average reward curve: During the initial 200 episodes, the policy engaged primarily in random exploration, resulting in substantial reward fluctuations near zero. Between episodes 200 and 350, guided by the advantage function, the policy distribution converged toward optimal trajectories, causing rapid reward growth. From episodes 350 to 600, the clipped probability mechanism constrained excessive gradient updates, leading to a transient reward decrease. Beyond episode 600, entropy regularization stabilized the reward curve, maintaining high values with minimal oscillations.

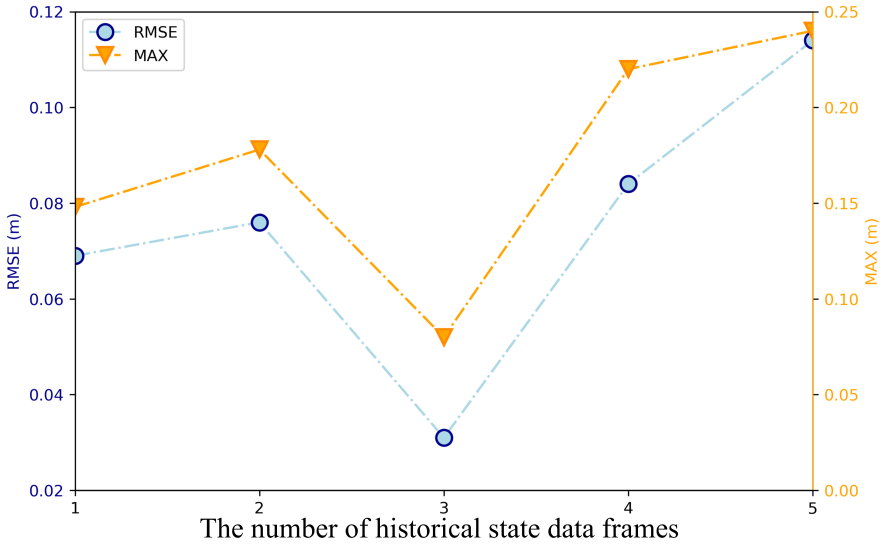

Figure 6. Comparison of the quadrotor's position between the reference and the predictions.

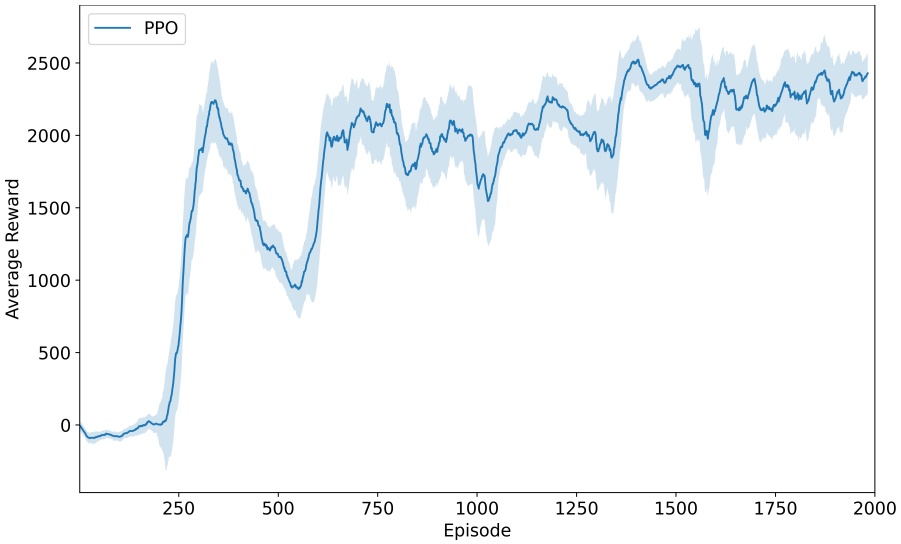

Figure 7. Training reward progression under wind-free conditions.

To quantitatively validate the pretrained PPO policy, Figure 8 compares reference and actual trajectories in a disturbance-free environment. The quadrotor tracked a circular path with radius = 1 m from initial position (1, 0, 3) m at a constant linear velocity

(1 m/s). As shown in Figure 8, the 3D trajectory plot explicitly compares the reference and prediction with the foundation model. As demonstrated in Figures 9 and 10, the foundation model achieves high-precision trajectory tracking in wind-free environments. The policy achieves precise tracking with near-perfect circular closure, indicating negligible cumulative drift. Maximum deviations occur at curvature maxima where centrifugal forces peak, yet all-axis errors remain below 0.05 m, satisfying precision thresholds for quadrotor maneuvering tasks.

This result empirically verifies that the PPO-derived policy converges to a robust nominal controller under ideal conditions, establishing a foundational model for subsequent continual learning.

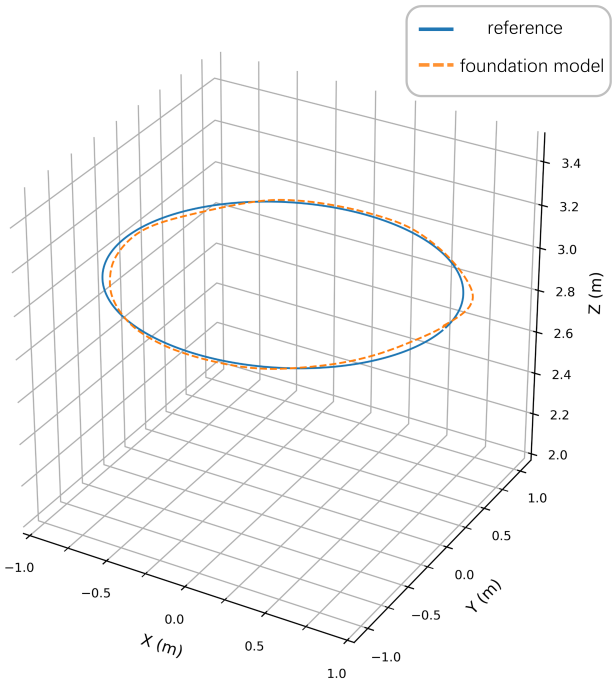

Figure 8. Reference versus actual trajectory in wind-free environment.

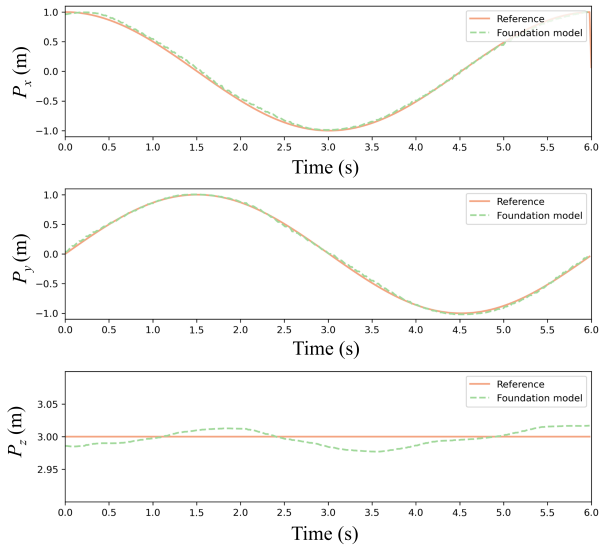

Figure 9. Comparison of the results of the foundation model and reference trajectory.

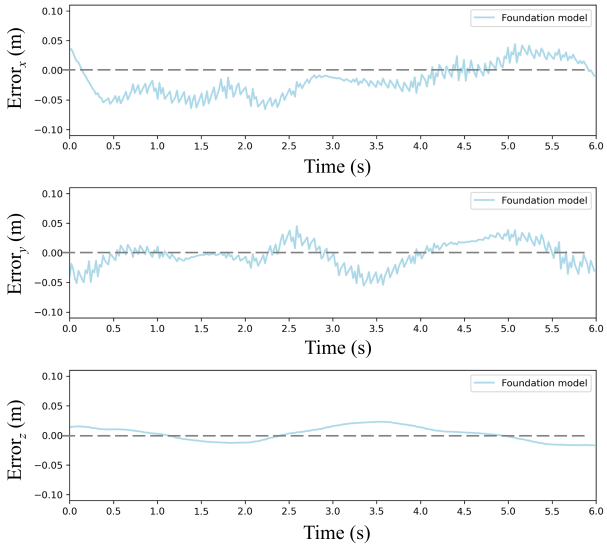

Figure 10. Error of the foundation model.

4.3. Performance Analysis in the Wind-Disturbed Environment

4.3.1. Performance in Stepwise Wind

To evaluate the continual learning capability of our algorithm in dynamic wind fields, we constructed a stepwise wind variation scenario: after 2000 episodes of foundation model training, a wind speed increment of 7 m/s was applied every 200 episodes along both x- and y-axes, creating a progressively intensifying wind field. Figure 11 compares the average reward curves of PPO and our continual reinforcement learning approach, with the horizontal axis representing training episodes and the vertical axis the average reward. Experimental results demonstrate that during low-wind phases, both methods exhibit comparable oscillation amplitudes while maintaining high returns. As wind speed progressively increases, the PPO strategy shows continuous reward degradation, indicating insufficient disturbance adaptability. In contrast, the CBP-enhanced policy exhibits transient reward drops after each wind speed change but rapidly recovers to prior levels, maintaining stable oscillations thereafter. The characteristic sawtooth reward pattern signifies substantially enhanced network plasticity, confirming the algorithm's continual learning and rapid adaptation capabilities in dynamic environments.

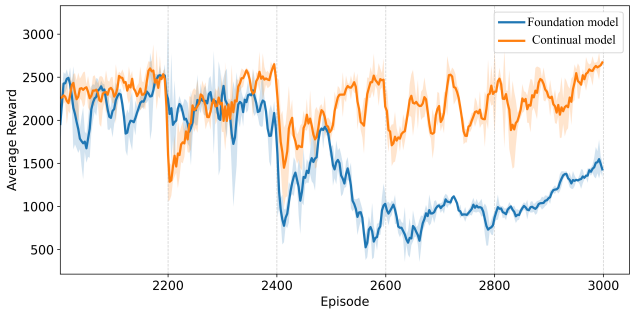

Figure 11. Average reward comparison under stepwise wind disturbances.

Figures 12 and 13 present trajectory comparisons under extreme wind conditions (cumulative 35 m/s along x/y-axes). Figure 12 displays reference versus controlled trajectories in three spatial dimensions, while Figure 13 shows corresponding positional errors. The results indicate that our continual reinforcement learning achieves lower error peaks than PPO at each wind speed increment while maintaining faster recovery to foundation performance. Overall average error is reduced by approximately 15%, validating the CBP

algorithm's significant enhancement of policy robustness and precision. Critically, the continual reinforcement learning method maintains accurate path tracking under strong disturbances, whereas standard PPO exhibits substantial deviation.

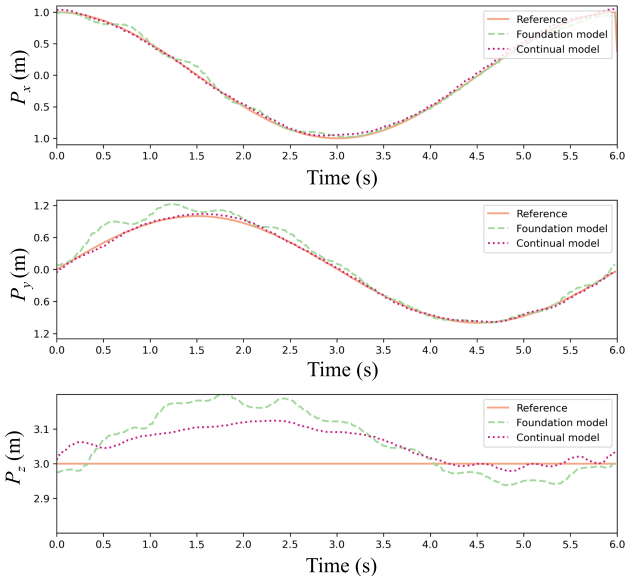

Figure 12. Trajectory comparison of the foundation model and continual model under stepwise wind disturbances.

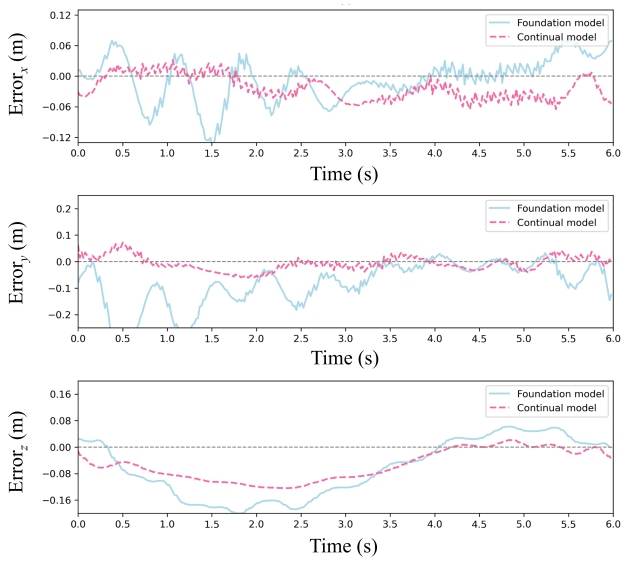

Figure 13. Error of the foundation model and continual model under stepwise wind disturbances.

These findings demonstrate that CBP's selective neuron resetting and structural diversity preservation empower reinforcement learning policies to maintain efficacy in evolving wind fields, significantly enhancing quadrotor adaptation to novel conditions.

4.3.2. Performance Under Stochastic Wind

To validate the adaptability of the proposed continual reinforcement learning framework in realistic wind environments, we conducted additional trajectory tracking experiments in a stochastic wind scenario. As depicted in Figure 14, random sampling of wind speeds in the X-axis $v_{w,x}$ and Y-axis $v_{w,y}$ was performed every 10 episodes (approximately 1 min). Both $v_{w,x}$ and $v_{w,y}$ were subject to a normal disturbance distribution with parameters $\mathcal{N}(4, 2.5)$. The trajectory reference entailed a circular path with a radius of 1 m starting

from the initial position at (1, 0, 3) m, moving at a constant linear velocity of 1 m/s. The evaluation criterion remained the RMSE of positional tracking, comparing two methods: the foundational model and the proposed continual reinforcement learning model. The training duration spanned 100 episodes to observe long-term adaptation trends.

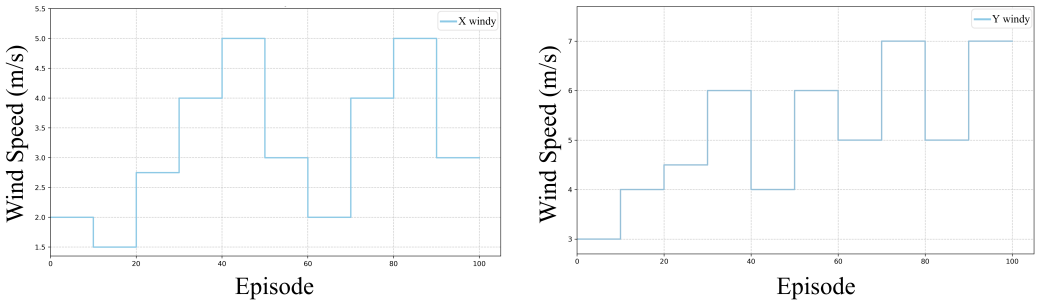

Figure 14. Stochastic wind speed variations.

Figure 15 illustrates the RMSE of the continual model training in a stochastic wind field over 100 episodes. The continual model exhibits notable stability throughout the training period. The RMSE values consistently remain low, predominantly below 0.06 m, indicating the efficacy of the CBP mechanism. This mechanism's capacity to dynamically reset inefficient neurons enables the model to adjust to varying stochastic wind conditions without experiencing catastrophic forgetting. Consequently, when confronted with new wind patterns, the model promptly reconfigures its neural network to identify fresh optimal policies while retaining valuable insights from prior wind conditions.

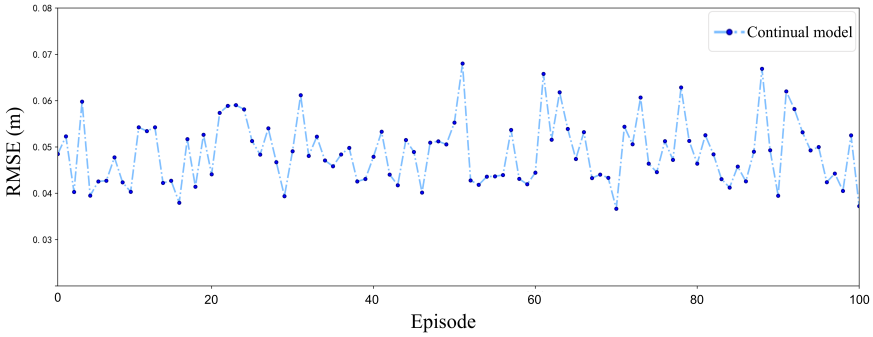

Figure 15. RMSE of continual model under stochastic wind disturbances.

Figures 16 and 17 present a comparative analysis of the foundation model and the continual model in a single episode under random wind conditions. Figure 16 depicts trajectory comparisons between the two models, showing that the continual model closely aligns with the reference trajectory in the three directions, even amidst significant random wind disturbances. Further insights from Figure 17, focusing on error dynamics, reveal that the continual model maintains positional errors within ±0.06 m across all three directions, swiftly recovering from transient wind gusts. In contrast, the foundation model exhibits persistent error accumulation, particularly evident in y-direction errors exceeding ±0.2 m in later episodes. This disparity arises from the utility-based neuron management of CBP, wherein underperforming neurons are reset to explore adaptive strategies in response to new disturbance patterns induced by random winds while retaining valuable features learned from prior wind conditions.

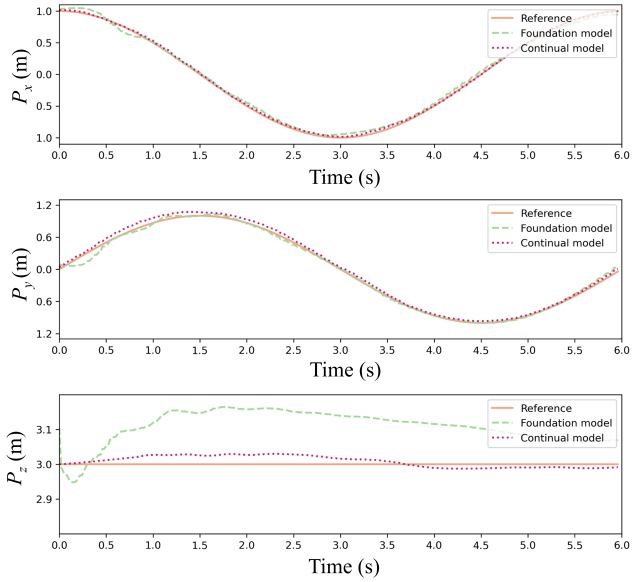

Figure 16. Trajectory comparison between the foundation model and continual model under stochastic wind disturbances.

Stochastic wind disturbances emulate natural atmospheric conditions more accurately, featuring unstructured variations in magnitude and direction. This necessitates enhanced real-time adaptation and robustness from the controller. The outcomes in the stochastic scenario validate the superior performance of the suggested framework over the conventional PPO model in terms of real-time adaptability and robustness.

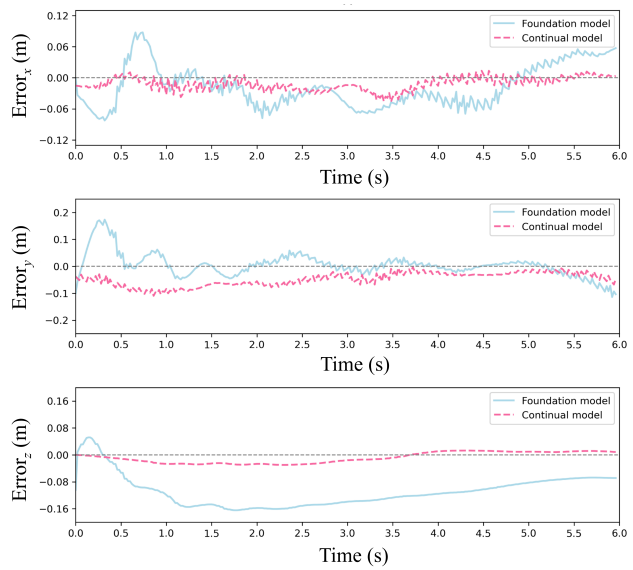

Figure 17. Error of the foundation model and continual model under stochastic wind disturbances.

5. Conclusions

This study establishes a high-precision quadrotor trajectory tracking foundation model based on the proximal policy optimization algorithm and then introduces the continual backpropagation algorithm to propose a continual reinforcement learning framework for dynamic wind-disturbed environments. Simulation results demonstrate that compared to standard PPO, our continual reinforcement learning approach achieves lower tracking errors under multiple wind speed increments. These findings validate the significant

enhancement of policy precision and robustness through continual learning mechanisms. Future work will explore CBP applications in real-flight validation, cross-task transfer learning, and more complex mission scenarios to enhance the generalization and cross-task adaptation capabilities of UAV reinforcement learning strategies.

Author Contributions: Methodology, Y.L.; Formal analysis, Y.L.; Resources, X.W.; Writing—original draft, Y.L.; Visualization, S.W.; Supervision, S.W.; Funding acquisition, L.H. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China under grant number 62461160260.

Data Availability Statement: The original contributions presented in this study are included in the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Chen, C.; Zheng, Z.; Xu, T.; Guo, S.; Feng, S.; Yao, W.; Lan, Y. Yolo-based uav technology: A review of the research and its applications. *Drones* **2023**, *7*, 190. [CrossRef]
2. Wang, J.; Alattas, K.A.; Bouteraa, Y.; Mofid, O.; Mobayen, S. Adaptive finite-time backstepping control tracker for quadrotor UAV with model uncertainty and external disturbance. *Aerosp. Sci. Technol.* **2023**, *133*, 108088. [CrossRef]
3. Buchelt, A.; Adrowitzer, A.; Kieseberg, P.; Gollob, C.; Nothdurft, A.; Eresheim, S.; Tschiatschek, S.; Stampfer, K.; Holzinger, A. Exploring artificial intelligence for applications of drones in forest ecology and management. *For. Ecol. Manag.* **2024**, *551*, 121530. [CrossRef]
4. Messaoudi, K.; Oubbati, O.S.; Rachedi, A.; Lakas, A.; Bendouma, T.; Chaib, N. A survey of UAV-based data collection: Challenges, solutions and future perspectives. *J. Netw. Comput. Appl.* **2023**, *216*, 103670. [CrossRef]
5. Tarekegn, G.B.; Juang, R.T.; Lin, H.P.; Munaye, Y.Y.; Wang, L.C.; Bitew, M.A. Deep-reinforcement-learning-based drone base station deployment for wireless communication services. *IEEE Internet Things J.* **2022**, *9*, 21899–21915. [CrossRef]
6. Zhao, B.; Xian, B.; Zhang, Y.; Zhang, X. Nonlinear robust adaptive tracking control of a quadrotor UAV via immersion and invariance methodology. *IEEE Trans. Ind. Electron.* **2014**, *62*, 2891–2902. [CrossRef]
7. Idrissi, M.; Salami, M.; Annaz, F. A review of quadrotor unmanned aerial vehicles: Applications, architectural design and control algorithms. *J. Intell. Robot. Syst.* **2022**, *104*, 22. [CrossRef]
8. Mohsan, S.A.H.; Khan, M.A.; Noor, F.; Ullah, I.; Alsharif, M.H. Towards the unmanned aerial vehicles (UAVs): A comprehensive review. *Drones* **2022**, *6*, 147. [CrossRef]
9. Zhang, G.; Xing, Y.; Zhang, W.; Li, J. Prescribed Performance Control for USV-UAV via a Robust Bounded Compensating Technique. *IEEE Trans. Control Netw. Syst.* **2025**, 1–11. [CrossRef]
10. Xing, Z.; Zhang, Y.; Su, C.Y. Active wind rejection control for a quadrotor UAV against unknown winds. *IEEE Trans. Aerosp. Electron. Syst.* **2023**, *59*, 8956–8968. [CrossRef]
11. Ahangar, A.R.; Ohadi, A.; Khosravi, M.A. A novel firefighter quadrotor UAV with tilting rotors: Modeling and control. *Aerosp. Sci. Technol.* **2024**, *151*, 109248. [CrossRef]
12. Lopez-Sanchez, I.; Moreno-Valenzuela, J. PID control of quadrotor UAVs: A survey. *Annu. Rev. Control* **2023**, *56*, 100900. [CrossRef]
13. Okasha, M.; Kralev, J.; Islam, M. Design and experimental comparison of pid, lqr and mpc stabilizing controllers for parrot mambo mini-drone. *Aerospace* **2022**, *9*, 298. [CrossRef]
14. Chen, L.; Liu, Z.; Gao, H.; Wang, G. Robust adaptive recursive sliding mode attitude control for a quadrotor with unknown disturbances. *ISA Trans.* **2022**, *122*, 114–125. [CrossRef] [PubMed]
15. Cai, X.; Zhu, X.; Yao, W. Fixed-time trajectory tracking control of a quadrotor UAV under time-varying wind disturbances: Theory and experimental validation. *Meas. Sci. Technol.* **2024**, *35*, 086205. [CrossRef]
16. Xu, Z.; Fan, L.; Qiu, W.; Wen, G.; He, Y. A robust disturbance-rejection controller using model predictive control for quadrotor UAV in tracking aggressive trajectory. *Drones* **2023**, *7*, 557. [CrossRef]
17. Zhao, W.; Liu, H.; Lewis, F.L. Data-driven fault-tolerant control for attitude synchronization of nonlinear quadrotors. *IEEE Trans. Autom. Control* **2021**, *66*, 5584–5591. [CrossRef]
18. Tan, J.; Xue, S.; Guo, Z.; Li, H.; Cao, H.; Chen, B. Data-driven optimal shared control of unmanned aerial vehicles. *Neurocomputing* **2025**, *622*, 129428. [CrossRef]

19. Park, S.; Park, C.; Kim, J. Learning-based cooperative mobility control for autonomous drone-delivery. *IEEE Trans. Veh. Technol.* **2023**, *73*, 4870–4885. [CrossRef]
20. Oh, Y.; Lee, M.H.; Moon, J. Koopman-Based Control System for Quadrotors in Noisy Environments. *IEEE Access* **2024**, *12*, 71675–71684. [CrossRef]
21. Pi, C.H.; Ye, W.Y.; Cheng, S. Robust quadrotor control through reinforcement learning with disturbance compensation. *Appl. Sci.* **2021**, *11*, 3257. [CrossRef]
22. Liu, P.; Ye, R.; Shi, K.; Yan, B. Full backstepping control in dynamic systems with air disturbances optimal estimation of a quadrotor. *IEEE Access* **2021**, *9*, 34206–34220. [CrossRef]
23. Lu, S.; Li, Y.; Liu, Z. Quadrotor Control using Reinforcement Learning under Wind Disturbance. In Proceedings of the 2023 35th Chinese Control and Decision Conference (CCDC), Yichang, China, 20–22 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 3233–3240.
24. Sacco, A.; Esposito, F.; Marchetto, G.; Montuschi, P. Sustainable task offloading in UAV networks via multi-agent reinforcement learning. *IEEE Trans. Veh. Technol.* **2021**, *70*, 5003–5015. [CrossRef]
25. Ma, B.; Liu, Z.; Dang, Q.; Zhao, W.; Wang, J.; Cheng, Y.; Yuan, Z. Deep reinforcement learning of UAV tracking control under wind disturbances environments. *IEEE Trans. Instrum. Meas.* **2023**, *72*, 2510913. [CrossRef]
26. Liu, H.; Li, B.; Xiao, B.; Ran, D.; Zhang, C. Reinforcement learning-based tracking control for a quadrotor unmanned aerial vehicle under external disturbances. *Int. J. Robust Nonlinear Control* **2023**, *33*, 10360–10377. [CrossRef]
27. Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; Veness, J.; Desjardins, G.; Rusu, A.A.; Milan, K.; Quan, J.; Ramalho, T.; Grabska-Barwinska, A.; et al. Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci. USA* **2017**, *114*, 3521–3526. [CrossRef]
28. Zenke, F.; Poole, B.; Ganguli, S. Continual learning through synaptic intelligence. In Proceedings of the International Conference on Machine Learning, PMLR, Sydney, Australia, 6–11 August 2017; pp. 3987–3995.
29. Li, Z.; Hoiem, D. Learning without forgetting. *IEEE Trans. Pattern Anal. Mach. Intell.* **2017**, *40*, 2935–2947. [CrossRef]
30. Dohare, S.; Hernandez-Garcia, J.F.; Lan, Q.; Rahman, P.; Mahmood, A.R.; Sutton, R.S. Loss of plasticity in deep continual learning. *Nature* **2024**, *632*, 768–774. [CrossRef]
31. Xie, J.; Peng, X.; Wang, H.; Niu, W.; Zheng, X. UAV autonomous tracking and landing based on deep reinforcement learning strategy. *Sensors* **2020**, *20*, 5630. [CrossRef]
32. Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; Meger, D. Deep reinforcement learning that matters. In Proceedings of the AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018; Volume 32.
33. Islam, R.; Henderson, P.; Gomrokchi, M.; Precup, D. Reproducibility of benchmarked deep reinforcement learning tasks for continuous control. *arXiv* **2017**, arXiv:1708.04133. [CrossRef]
34. Kingma, D.P.; Ba, J. Adam: A method for stochastic optimization. *arXiv* **2014**, arXiv:1412.6980.
35. Macenski, S.; Foote, T.; Gerkey, B.; Lalancette, C.; Woodall, W. Robot operating system 2: Design, architecture, and uses in the wild. *Sci. Robot.* **2022**, *7*, eabm6074. [CrossRef]
36. Ma, B.; Liu, Z.; Zhao, W.; Yuan, J.; Long, H.; Wang, X.; Yuan, Z. Target tracking control of UAV through deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 5983–6000. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.